

Traffic Sign Classifier

Team ID: SDCC2021001

Presenter: Krushna Borude

PROJECT - 2
REPORT

Submitted by: Team Avida of *National Institute of Technology Agartala*

For: SAE Self Driving Car Challenge 2022

DECLARATION

I the undersigned solemnly declare that the project report titled TRAFFIC SIGN CLASSIFICATION is based on my own work carried out during the course of our study under the supervision of Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section. I assert the statements made and conclusions drawn are an outcome of my research work. I further certify that,

I. The work contained in the report is original and has been done by me under the general supervision of my supervisor.

II. The work has not been submitted to any other institution for any other degree/diploma/certificate in this university or any other university of India or abroad.

III. I have followed the guidelines provided by the university in writing the report.

IV. Whenever I have used materials(data, theoretical analysis and text)from other sources, I have given due credit to them in the references.

Keerthana Neigapula

Hareesh M

Shivansh Mishra

Dikshita Jaraut

Sowjanya Matakala

Sumit Anand

Anawil Tiwari

Krushnakant Borude

Thanesh Bisen

Rakshit Gupta Date –

09/06/2022

ACKNOWLEDGEMENTS

I would like to use this opportunity to thank my mentor Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section. for guiding me throughout the project duration. I would also like to thank SAE India Southern Section for providing me an opportunity to work on this interesting project.

I owe gratitude to various books and online resourcesfor the completion of this project. These resources have been mentioned in the references.

Contents

1. Objective
2. Introduction
3. What is traffic sign classification?
4. Traffic Sign Dataset
5. Challenges with the GTSRB dataset
6. Packages and libraries
7. Methodology
8. CODE
9. Future Scope
10. Conclusion
11. References

Traffic Signal Classifier

Objective:

Discussing and implementing the concept of traffic sign classification and recognition, using a dataset.

Reviewing our directory structure for the project.

Implementation of TrafficSignNet, a Convolutional Neural Network which will help in training using our dataset.

Given our trained model, evaluating its accuracy on the test data and even learn how to make predictions on new input data as well.

Hence our major aim is to build a Road Sign Classifier in Keras or writing a CNN that classifies over 43 types of road signs in the Keras framework.

Introduction:

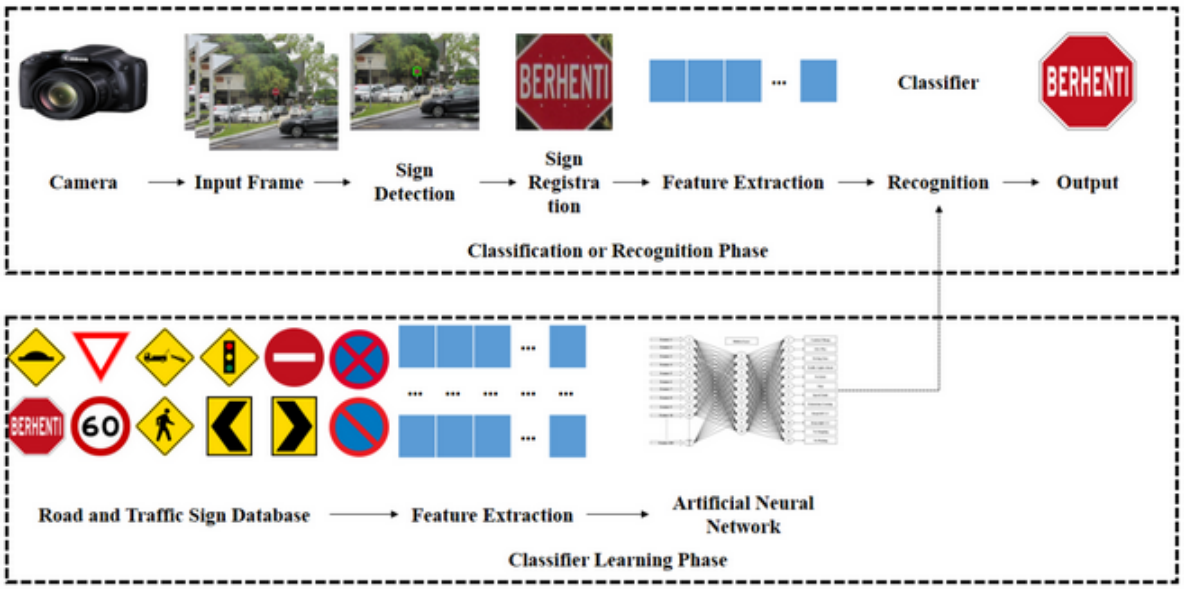
There are so many different types of traffic signs out there, each with different colours, shapes and sizes. Sometimes, there are two signs may have a similar colour, shape and size, but have 2 totally different meanings. To be able to program a computer to correctly classify a traffic sign on the road we creat our very own CNN to classify each different road sign.

What is traffic sign classification?

Traffic sign classification is the process of automatically recognizing traffic signs along the road, including speed limit signs, yield signs, merge signs, etc. Being able to automatically recognize traffic signs enables us to build “smarter cars”.

Self-driving cars need traffic sign recognition in order to properly parse and understand the roadway. Similarly, “driver alert” systems inside cars need to understand the roadway around them to help aid and protect drivers.

Traffic sign recognition is just one of the problems that computer vision and deep learning can solve.



Traffic sign dataset

The GTSRB (German Traffic Sign Recognition Benchmark) dataset, a dataset with over 50,000 images of German Traffic Signs was used. The GTSRB dataset consists of **43 traffic sign classes** and **nearly 50,000 images.**



The (GTSRB) dataset will be used for traffic sign classification with Keras and deep learning.

The traffic signs have been *pre-cropped* , implying that the dataset annotators/creators have manually labeled the signs in the images *and* extracted the traffic sign Region of Interest (ROI) for us, thereby simplifying the project.

The *meta* folder should have 43 different images (ranging from 0–42). The *test* folder is just a bunch of test images. The *train* folder should have 43 folders (again, ranging from 0–42), each containing images from its respective class.

In the real-world, traffic sign recognition is a two-stage process:

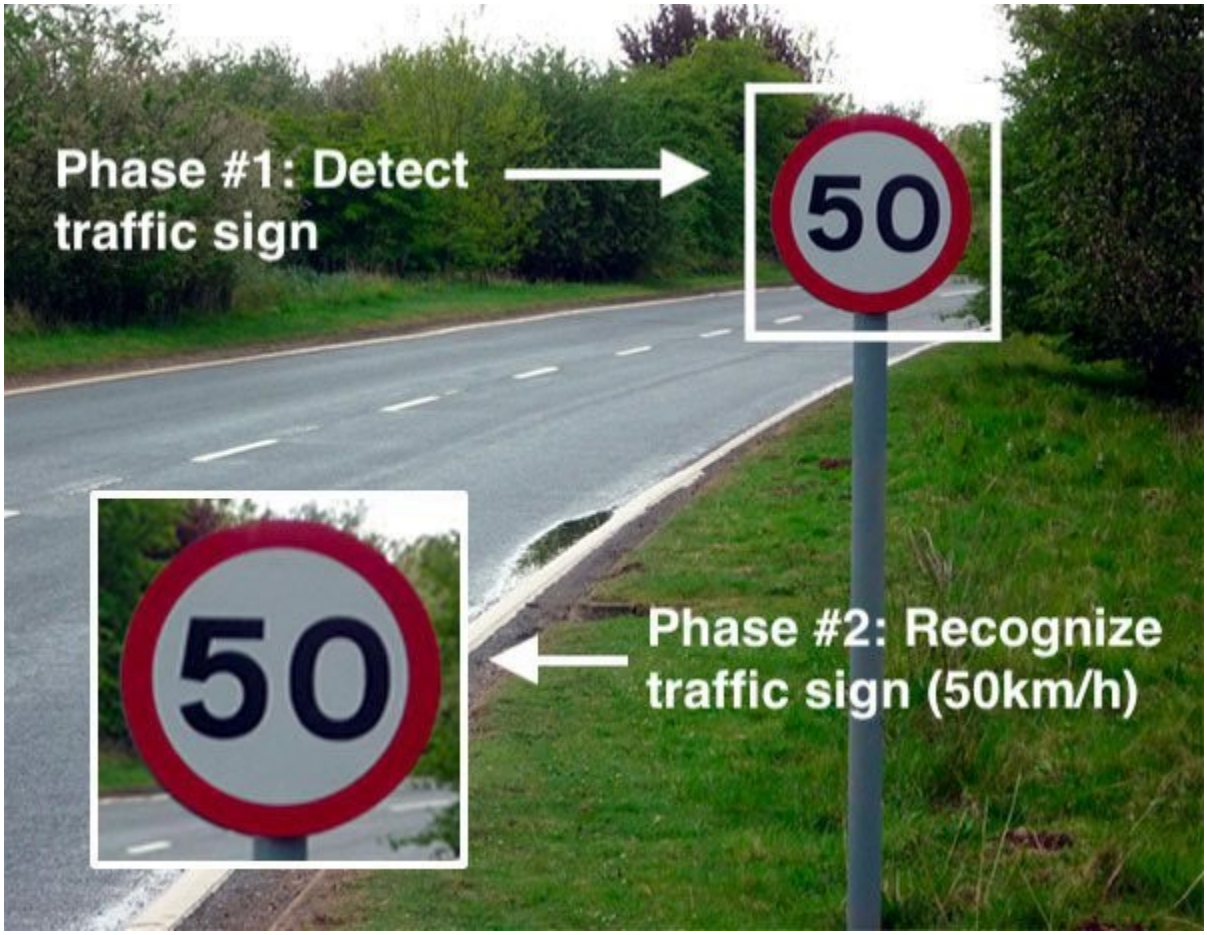
1. Localization: Detect and localize where in an input image/frame a traffic sign is.
2. Recognition: Take the localized ROI and actually *recognize* and *classify* the traffic sign.

Traffic sign recognition consists of object detection:

(1) detection/localization and

(2) classification.

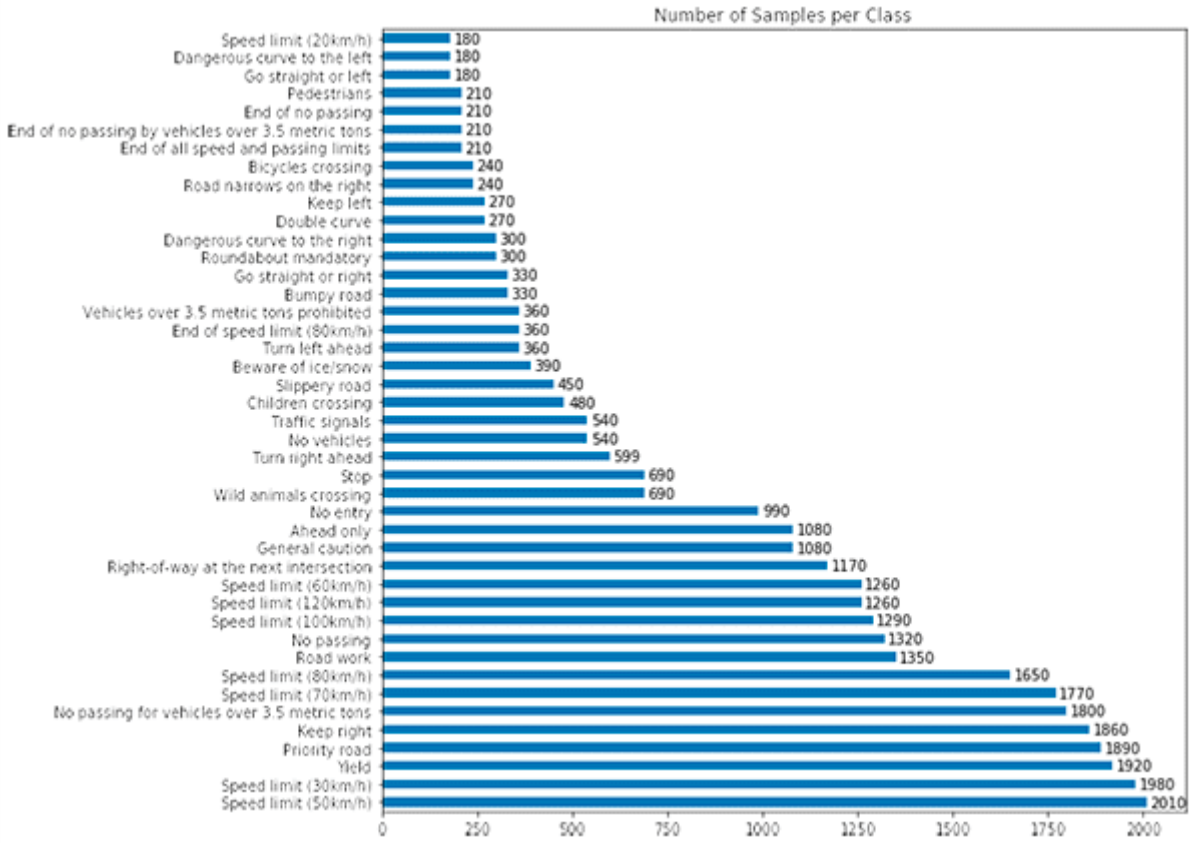
Deep learning object detectors can perform localization and recognition in a single forward-pass of the network .



Challenges with the GTSRB dataset

There are a number of challenges in the GTSRB dataset, **the first being that images are low resolution**, and worse, **have poor contrast**. These images are pixelated, and in some cases, it's extremely challenging, if not impossible, for the human eye and brain to recognize the sign.

The second challenge with the dataset is **handling class skew:**



The German Traffic Sign Recognition Benchmark (GTSRB) dataset is an example of an unbalanced dataset. We will account for this when training our traffic sign classifier with Keras and deep learning.

The top class *(Speed limit 50km/h)* has over 2,000 examples while the least represented class *(Speed limit 20km/h)* has under 200 examples — that's an order of magnitude difference!

Packages and libraries:

We at first need to make sure we have all the software packages installed. The libraries and packages used in this project are as follows:

1. NumPy

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

a. NumPy can be installed with conda, with pip, with a package manager on macOS and Linux, or from source
b. If you use pip, you can install NumPy with:
   pip install numpy
c. Also when using pip, it's good practice to use a virtual environment - see Reproducible Installs below for why, and this guide for details on using virtual environments.
d. If your installation fails with the message below, see Troubleshooting ImportError.

2. Matplotlib

It is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK. There is also a procedural "pylab" interface based on a state machine (like OpenGL), designed to closely resemble that of MATLAB, though its use is discouraged. SciPy makes use of Matplotlib.
John D. Hunter originally wrote Matplotlib. Since then it has an active development community and is distributed under a BSD-style license. Michael Droettboom was nominated as matplotlib's lead developer shortly before John Hunter's death in August 2012 and was further joined by Thomas Caswell. Matplotlib is a NumFOCUS fiscally sponsored project.

Matplotlib 2.0.x supports Python versions 2.7 through 3.10. Python 3 support started with Matplotlib 1.2. Matplotlib 1.4 is the last version to support Python 2.6. Matplotlib has pledged not to support Python 2 past 2020 by signing the Python 3 Statement.

a. Matplotlib releases are available as wheel packages for macOS, Windows and Linux on PyPI. Install it using pip:
   python -m pip install -U pip
   python -m pip install -U matplotlib
b. If this command results in Matplotlib being compiled from source and there's trouble with the compilation, you can add *--prefer-binary* to select the newest version of Matplotlib for which there is a precompiled wheel for your OS and Python.

3. Pandas

Pandas is an open-source library that is made mainly for working with relational or labeled data both easily and intuitively. It provides various data structures and operations for manipulating numerical data and time series. This library is built on top of the NumPy library. Pandas is fast and it has high performance & productivity for users.

4. Tensorflow

TensorFlow is an end-to-end open source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries and community resources that lets researchers push the state-of-the-art in ML and developers easily build and deploy ML powered applications. TensorFlow offers multiple levels of abstraction so you can choose the right one for your needs. TensorFlow has always provided a direct path to production. Whether it's on servers, edge devices, or the web, TensorFlow lets you train and deploy your model easily, no matter what language or platform you use.

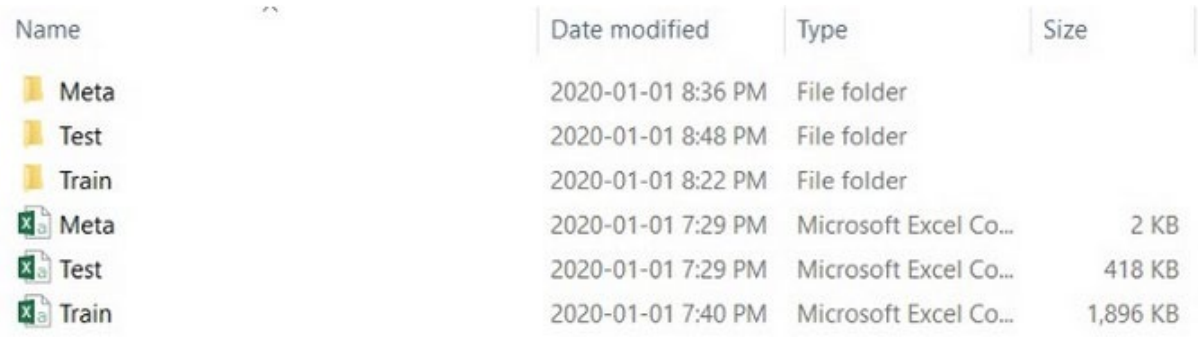
5. Keras

Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides. Built on top of TensorFlow 2, Keras is an industry-strength framework that can scale to large clusters of GPUs or an entire TPU pod. It's not only possible; it's easy. Take advantage of the full deployment capabilities of the TensorFlow platform. You can export Keras models to JavaScript to run directly in the browser, to TF Lite to run on iOS, Android, and embedded devices. It's also easy to serve Keras models as via a web API. Because of its ease-of-use and focus on user experience, Keras is the deep learning solution of choice for many university courses. It is widely recommended as one of the best ways to learn deep learning.

Methodology:

1.Importing and loading data

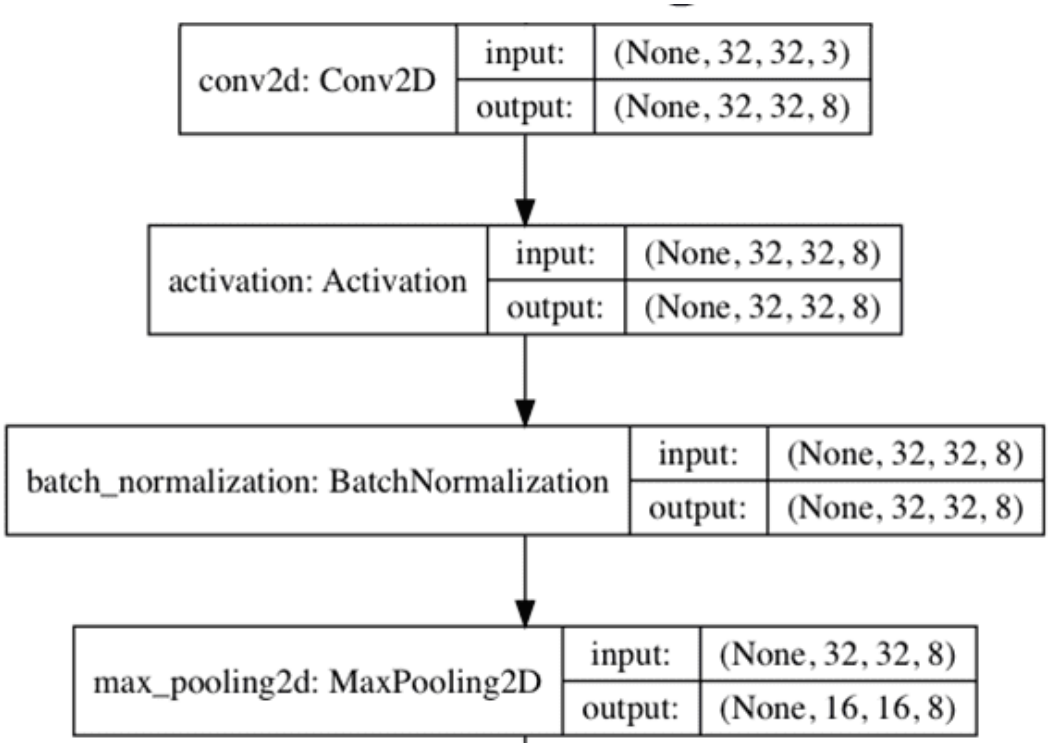
We are using the GTSRB dataset, a dataset with over 50,000 images of German Traffic Signs. There are 43 classes. When you open the dataset in your computer, there should be 6 paths inside your dataset (3 folders and 3 spreadsheets)



Name	Date modified	Type	Size
Meta	2020-01-01 8:36 PM	File folder	
Test	2020-01-01 8:48 PM	File folder	
Train	2020-01-01 8:22 PM	File folder	
Meta	2020-01-01 7:29 PM	Microsoft Excel Co...	2 KB
Test	2020-01-01 7:29 PM	Microsoft Excel Co...	418 KB
Train	2020-01-01 7:40 PM	Microsoft Excel Co...	1,896 KB

Firstly, we import the necessary libraries and modules that are required for us to load the data. The first bunch is libraries needed to create the *load_data* function. The second bunch is the stuff that we need to build our model. You can import each bunch in different kernels if you like. Our *load_data* function takes 1 parameter, which is the path to our dataset. After that, we define two lists, *images* and *classes*. The *images* list will store the image arrays and the *class* list will store the class number for each image. The final line randomizes our data which will prevent the model from overfitting to specific classes.

2.Implementing CNN traffic sign classifier



The Keras deep learning framework is used to build a Convolutional Neural Network (CNN) for traffic sign classification.

Here we implement a Convolutional Neural Network to classify and recognize traffic signs. This set of layers uses a *5×5* kernel to learn larger features — it will help to distinguish between different traffic sign shapes and color blobs on the traffic signs themselves. These sets of layers deepen the network by stacking two sets of CONV => RELU => BN layers before applying max-pooling to reduce volume dimensionality.

3. Implementing our training script

Now that our architecture has been implemented, we create our Python training script that will be responsible for:

- Loading our training and testing split from the GTSRB dataset
- Preprocessing the images
- Training our model
- Evaluating our model's accuracy
- Serializing the model to disk so we can later use it to make predictions on new traffic sign data

The GTSRB dataset is pre-split into training/testing splits for us. The format of the data is:

Width, Height, X1, Y1, X2, Y2, ClassID, Image Path.

Our three command line arguments consist of:

--dataset: The path to our GTSRB dataset.

--model: The desired path/filename of our output model.

--plot: The path to our training history plot.

4.Training our architecture on the traffic sign dataset

After training our dataset we will achieve results that can be validated using the accuracy obtained.

5.Implementing our prediction script

Now that our traffic sign recognition model is trained, we continue as follows:

1. Load the model from disk
2. Load sample images from disk
3. Preprocess the sample images in the same manner as we did for training
4. Pass our images through our traffic sign classifier
5. Obtain our final output predictions

6.Make predictions on traffic sign data

We finally make predictions on traffic sign data using our trained model.

CODE:

Importing Required Libraries

```
import os
Root =
"/content/drive/MyDrive/Colab_Notebook/TSF/Traffic_sign_classification"
os.chdir(Root)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
%tensorflow_version 1.x
```

```
TensorFlow 1.x selected.
```

```
!pip install tensorflow==1.14.0
import tensorflow as tf
print(tf.__version__)
```

```
import numpy as np
import pandas as pd
import os
import cv2
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from PIL import Image
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import accuracy_score
np.random.seed(42)

from matplotlib import style
style.use('fivethirtyeight')
```

Assigning Path for Dataset

```
data_dir =
"/content/drive/MyDrive/Colab_Notebook/TSF/Traffic_sign_classification"
train_path = 'Train'
test_path = 'Test'

# Resizing the images to 30x30x3
IMG_HEIGHT = 30
IMG_WIDTH = 30
channels = 3
```

Finding Total Classes

```
NUM_CATEGORIES = len(os.listdir(train_path))
NUM_CATEGORIES

Output: 43

# Label Overview
classes = { 0:'Speed limit (20km/h)',
            1:'Speed limit (30km/h)',
            2:'Speed limit (50km/h)',
            3:'Speed limit (60km/h)',
            4:'Speed limit (70km/h)',
            5:'Speed limit (80km/h)',
            6:'End of speed limit (80km/h)',
            7:'Speed limit (100km/h)',
            8:'Speed limit (120km/h)',
            9:'No passing',
            10:'No passing veh over 3.5 tons',
            11:'Right-of-way at intersection',
            12:'Priority road',
            13:'Yield',
            14:'Stop',
            15:'No vehicles',
            16:'Veh > 3.5 tons prohibited',
            17:'No entry',
            18:'General caution',
            19:'Dangerous curve left',
            20:'Dangerous curve right',
            21:'Double curve',
            22:'Bumpy road',
            23:'Slippery road',
            24:'Road narrows on the right',
            25:'Road work',
            26:'Traffic signals',
            27:'Pedestrians',
            28:'Children crossing',
            29:'Bicycles crossing',
            30:'Beware of ice/snow',
```

```
    31:'Wild animals crossing',
    32:'End speed + passing limits',
    33:'Turn right ahead',
    34:'Turn left ahead',
    35:'Ahead only',
    36:'Go straight or right',
    37:'Go straight or left',
    38:'Keep right',
    39:'Keep left',
    40:'Roundabout mandatory',
    41:'End of no passing',
    42:'End no passing veh > 3.5 tons' }
```

Visualizing The Dataset

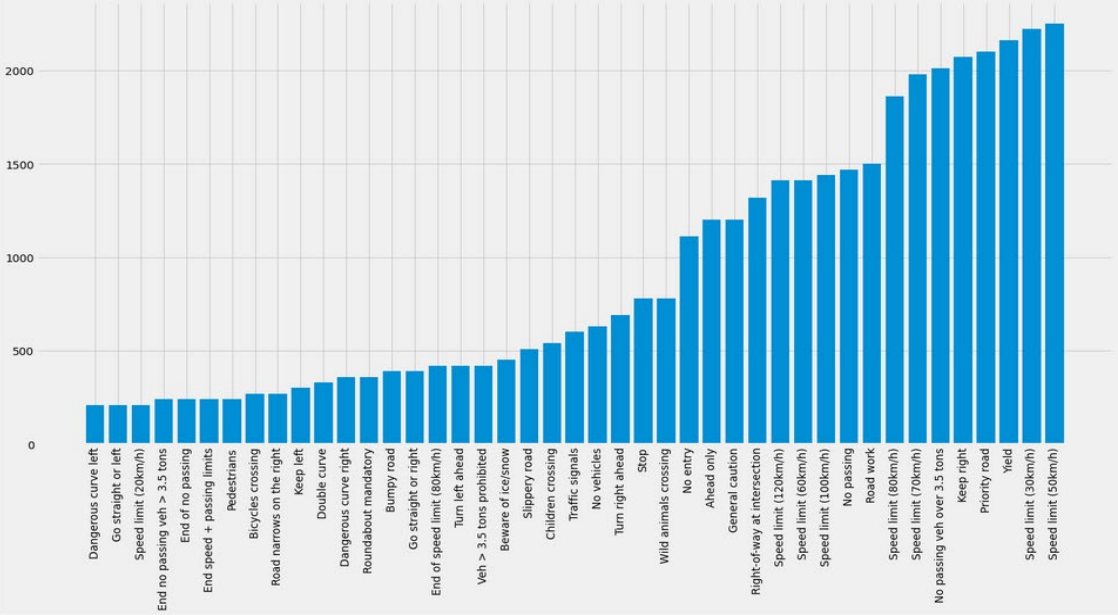
```
folders = os.listdir(train_path)

train_number = []
class_num = []

for folder in folders:
    train_files = os.listdir(train_path + '/' + folder)
    train_number.append(len(train_files))
    class_num.append(classes[int(folder)])

# Sorting the dataset on the basis of number of images in each class
zipped_lists = zip(train_number, class_num)
sorted_pairs = sorted(zipped_lists)
tuples = zip(*sorted_pairs)
train_number, class_num = [ list(tuple) for tuple in  tuples]

# Plotting the number of images in each class
plt.figure(figsize=(21,10))
plt.bar(class_num, train_number)
plt.xticks(class_num, rotation='vertical')
plt.show()
```



```
# Visualizing 25 random images from test data
import random
from matplotlib.image import imread

test = pd.read_csv(data_dir + '/Test.csv')
imgs = test["Path"].values

plt.figure(figsize=(25,25))

for i in range(1,26):
    plt.subplot(5,5,i)
    random_img_path = data_dir + '/' + random.choice(imgs)
    rand_img = imread(random_img_path)
    plt.imshow(rand_img)
    plt.grid(b=None)
    plt.xlabel(rand_img.shape[1], fontsize = 20)#width of image
    plt.ylabel(rand_img.shape[0], fontsize = 20)#height of image
```



Collecting the Training Data

```
image_data = []
image_labels = []

for i in range(NUM_CATEGORIES):
    path = data_dir + '/Train/' + str(i)
```

```
    images = os.listdir(path)

    for img in images:
        try:
            image = cv2.imread(path + '/' + img)
            image_fromarray = Image.fromarray(image, 'RGB')
            resize_image = image_fromarray.resize((IMG_HEIGHT, IMG_WIDTH))
            image_data.append(np.array(resize_image))
            image_labels.append(i)
        except:
            print("Error in " + img)

# Changing the list to numpy array
image_data = np.array(image_data)
image_labels = np.array(image_labels)

print(image_data.shape, image_labels.shape)

Output: (39209, 30, 30, 3) (39209,)
```

Shuffling the training data

```
shuffle_indexes = np.arange(image_data.shape[0])
np.random.shuffle(shuffle_indexes)
image_data = image_data[shuffle_indexes]
image_labels = image_labels[shuffle_indexes]
```

Splitting the data into train and validation set

```
X_train, X_val, y_train, y_val = train_test_split(image_data, image_labels,
test_size=0.3, random_state=42, shuffle=True)

X_train = X_train/255
X_val = X_val/255

print("X_train.shape", X_train.shape)
print("X_valid.shape", X_val.shape)
print("y_train.shape", y_train.shape)
print("y_valid.shape", y_val.shape)

Output:
X_train.shape (27446, 30, 30, 3)
X_valid.shape (11763, 30, 30, 3)
y_train.shape (27446,)
y_valid.shape (11763,)
```

One hot encoding the labels

```
y_train = keras.utils.to_categorical(y_train, NUM_CATEGORIES)
y_val = keras.utils.to_categorical(y_val, NUM_CATEGORIES)

print(y_train.shape)
print(y_val.shape)

Output:
(27446, 43)
(11763, 43)
```

Making the model

```
model = keras.models.Sequential([
    keras.layers.Conv2D(filters=16, kernel_size=(3,3), activation='relu',
input_shape=(IMG_HEIGHT,IMG_WIDTH,channels)),
    keras.layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu'),
    keras.layers.MaxPool2D(pool_size=(2, 2)),
    keras.layers.BatchNormalization(axis=-1),

    keras.layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    keras.layers.Conv2D(filters=128, kernel_size=(3,3), activation='relu'),
    keras.layers.MaxPool2D(pool_size=(2, 2)),
    keras.layers.BatchNormalization(axis=-1),

    keras.layers.Flatten(),
    keras.layers.Dense(512, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(rate=0.5),

    keras.layers.Dense(43, activation='softmax')
])


lr = 0.001
epochs = 30

opt = Adam(lr=lr, decay=lr / (epochs * 0.5))
model.compile(loss='categorical_crossentropy', optimizer=opt,
metrics=['accuracy'])
```

Augmenting the data and training the model

```
aug = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.15,
    horizontal_flip=False,
    vertical_flip=False,
    fill_mode="nearest")

history = model.fit(aug.flow(X_train, y_train, batch_size=32),
epochs=epochs, validation_data=(X_val, y_val))
```

```
Epoch 1/5
857/858 [============================>.] - ETA: 0s - loss: 1.0416 - acc: 0.7210Epoch 1/5
11765/858 [==========================================================================================================================================
858/858 [==============================] - 105s 123ms/step - loss: 1.0406 - acc: 0.7213 - val_loss: 0.0834 - val_acc: 0.9745
Epoch 2/5
857/858 [============================>.] - ETA: 0s - loss: 0.1555 - acc: 0.9539Epoch 1/5
11765/858 [==========================================================================================================================================
858/858 [==============================] - 106s 123ms/step - loss: 0.1554 - acc: 0.9539 - val_loss: 0.0310 - val_acc: 0.9911
Epoch 3/5
857/858 [============================>.] - ETA: 0s - loss: 0.0786 - acc: 0.9761Epoch 1/5
11765/858 [==========================================================================================================================================
858/858 [==============================] - 116s 135ms/step - loss: 0.0787 - acc: 0.9761 - val_loss: 0.0113 - val_acc: 0.9965
Epoch 4/5
857/858 [============================>.] - ETA: 0s - loss: 0.0529 - acc: 0.9846Epoch 1/5
11765/858 [==========================================================================================================================================
858/858 [==============================] - 105s 123ms/step - loss: 0.0530 - acc: 0.9846 - val_loss: 0.0084 - val_acc: 0.9975
Epoch 5/5
857/858 [============================>.] - ETA: 0s - loss: 0.0444 - acc: 0.9865Epoch 1/5
11765/858 [==========================================================================================================================================
858/858 [==============================] - 105s 123ms/step - loss: 0.0445 - acc: 0.9865 - val_loss: 0.0157 - val_acc: 0.9958
```

```
model.save("model.h5")
```

```
model.save_weights("/content/drive/MyDrive/Colab_Notebook/TSF/Traffic_sign_
classification/output")

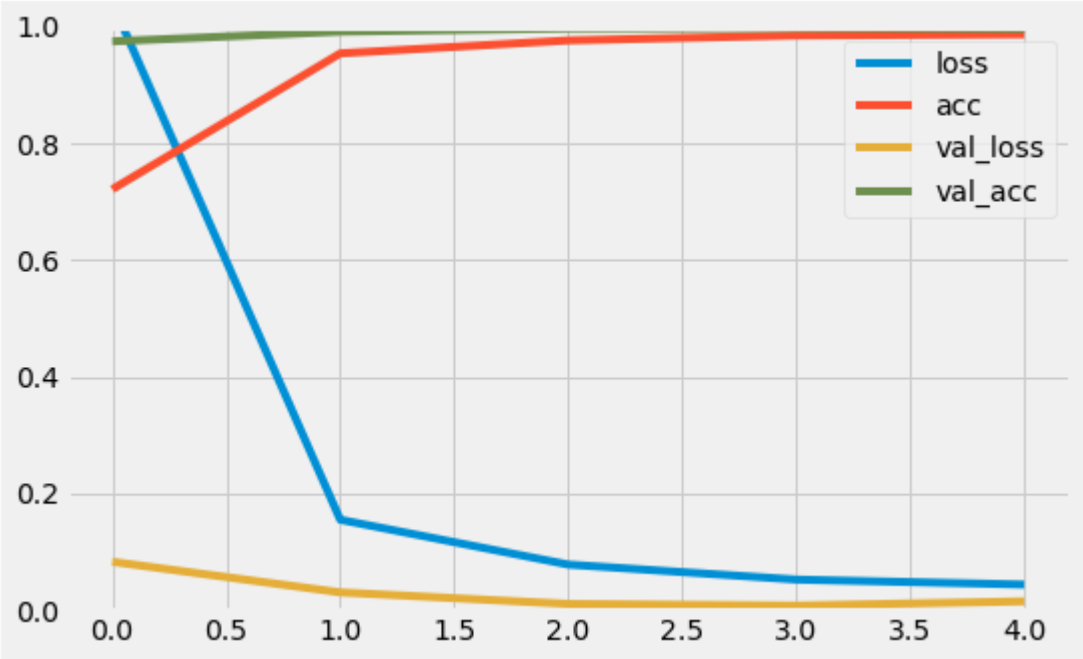
dir(aug)

Output:
['__class__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 '__weakref__',
 '_keras_api_names',
 '_keras_api_names_v1',
 '_validation_split',
 'apply_transform',
 'brightness_range',
 'channel_axis',
 'channel_shift_range',
 'col_axis',
 'cval',
 'data_format',
 'dtype',
 'featurewise_center',
 'featurewise_std_normalization',
 'fill_mode',
 'fit',
 'flow',
 'flow_from_dataframe',
 'flow_from_directory',
 'get_random_transform',
 'height_shift_range',
 'horizontal_flip',
 'interpolation_order',
 'mean',
 'preprocessing_function',
 'principal_components',
 'random_transform',
 'rescale',
 'rotation_range',
 'row_axis',
```

```
 'samplewise_center',
 'samplewise_std_normalization',
 'shear_range',
 'standardize',
 'std',
 'vertical_flip',
 'width_shift_range',
 'zca_epsilon',
 'zca_whitening',
 'zoom_range']
```

Evaluating the model

```
pd.DataFrame(history.history).plot(figsize=(8, 5))
plt.grid(True)
plt.gca().set_ylim(0, 1)
plt.show()
```



Loading the test data and running the predictions

```
test = pd.read_csv(data_dir + '/Test.csv')

labels = test["ClassId"].values
imgs = test["Path"].values

data =[]

for img in imgs:
    try:
        image = cv2.imread(data_dir + '/' +img)
        image_fromarray = Image.fromarray(image, 'RGB')
        resize_image = image_fromarray.resize((IMG_HEIGHT, IMG_WIDTH))
        data.append(np.array(resize_image))
    except:
        print("Error in " + img)
X_test = np.array(data)
X_test = X_test/255
```

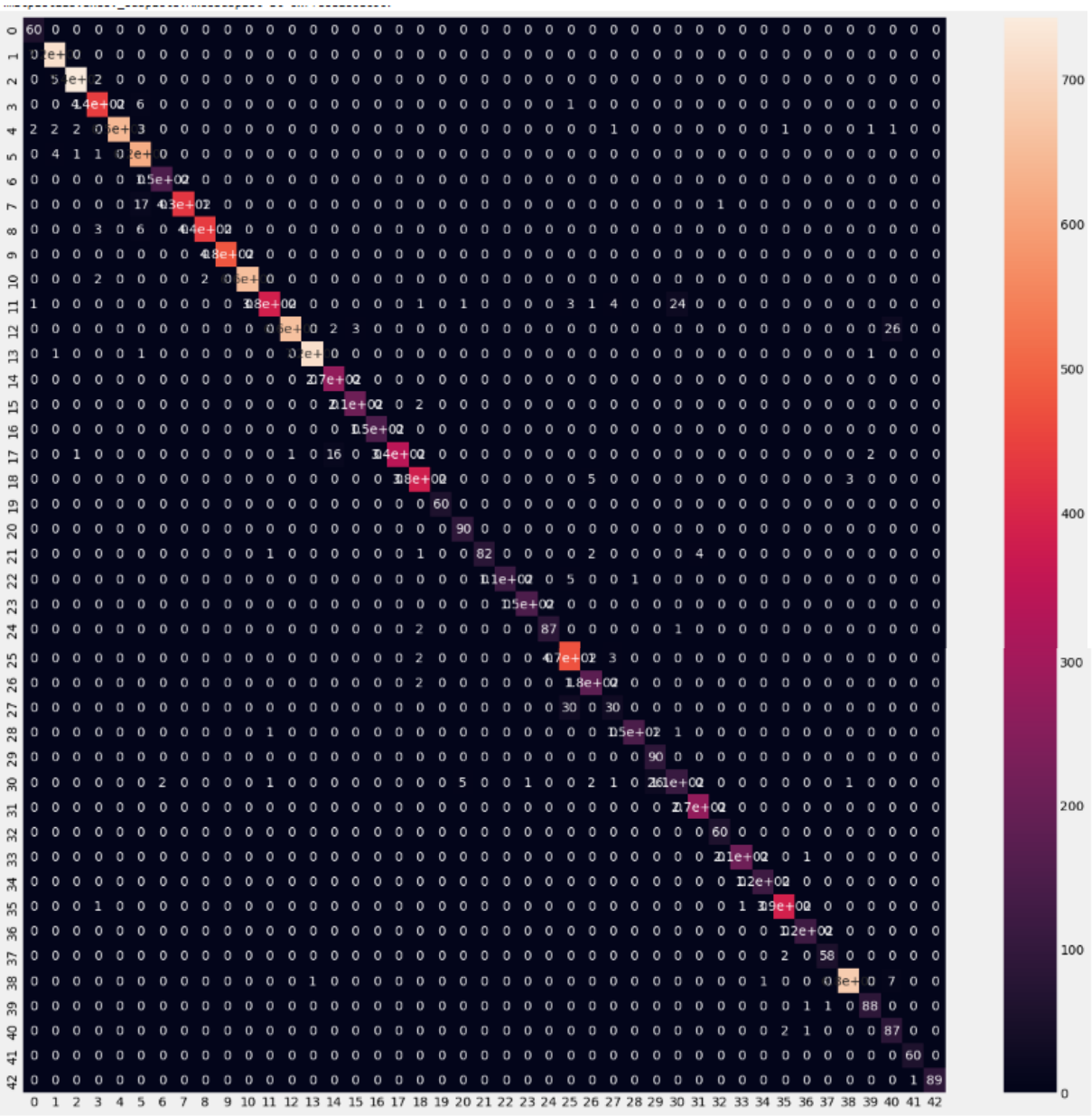
```
pred = model.predict_classes(X_test)

#Accuracy with the test data
print('Test Data accuracy: ',accuracy_score(labels, pred)*100)
```

Visualizing the confusion matrix

```
from sklearn.metrics import confusion_matrix
cf = confusion_matrix(labels, pred)

import seaborn as sns
df_cm = pd.DataFrame(cf, index = classes,  columns = classes)
plt.figure(figsize = (20,20))
sns.heatmap(df_cm, annot=True)
```



Classification report

```
from sklearn.metrics import classification_report

print(classification_report(labels, pred))
```

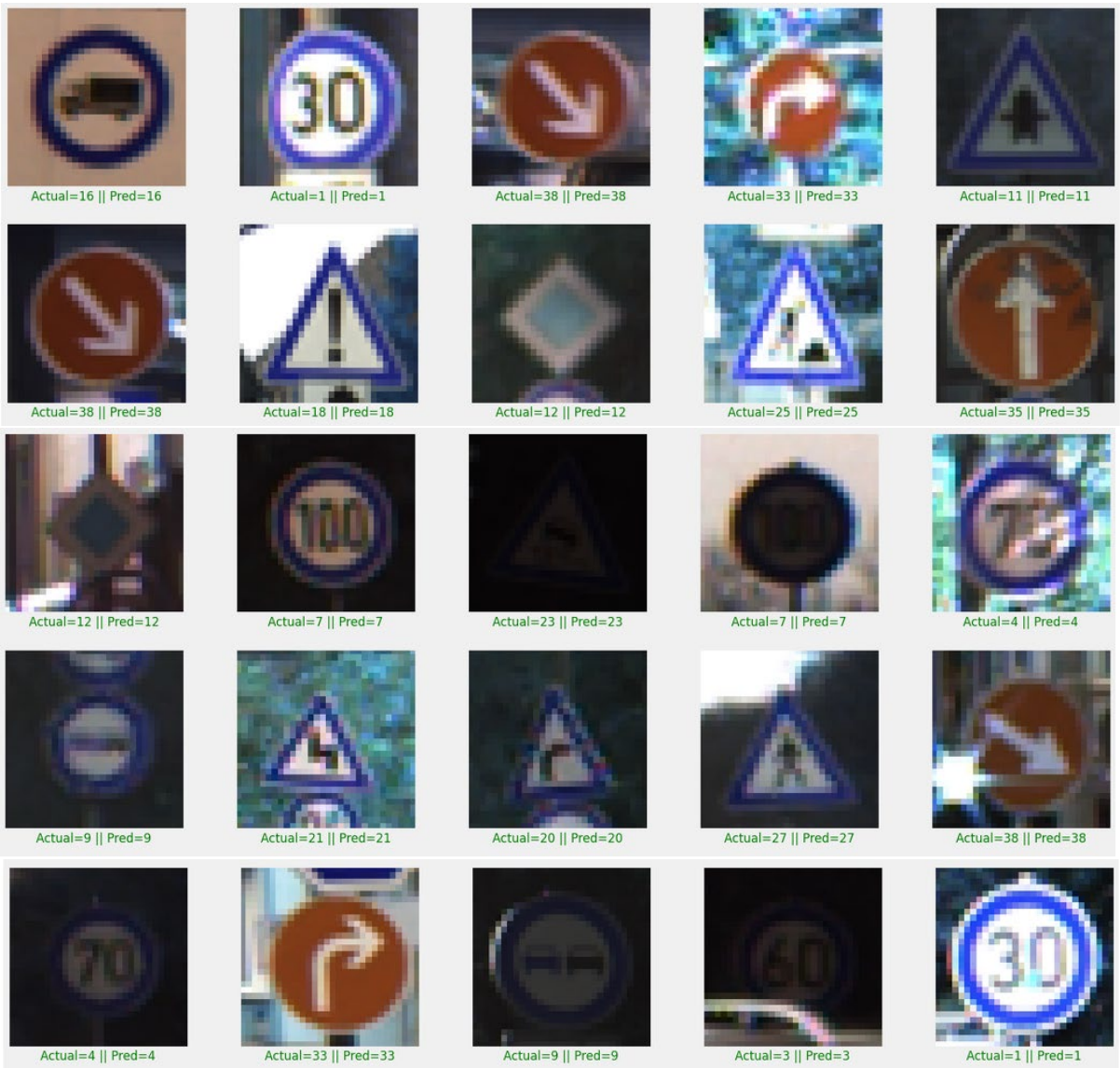
Output:

	precision	recall	f1-score	support
0	0.95	1.00	0.98	60
1	0.98	1.00	0.99	720
2	0.99	0.99	0.99	750
3	0.98	0.98	0.98	450
4	1.00	0.98	0.99	660
5	0.95	0.99	0.97	630
6	0.99	1.00	0.99	150
7	1.00	0.96	0.98	450
8	0.99	0.98	0.99	450
9	1.00	1.00	1.00	480
10	1.00	0.99	1.00	660
11	0.99	0.92	0.95	420
12	1.00	0.96	0.98	690
13	1.00	1.00	1.00	720
14	0.94	1.00	0.97	270
15	0.99	0.99	0.99	210
16	1.00	1.00	1.00	150
17	1.00	0.94	0.97	360
18	0.97	0.98	0.98	390
19	1.00	1.00	1.00	60
20	0.94	1.00	0.97	90
21	1.00	0.91	0.95	90
22	1.00	0.95	0.97	120
23	0.99	1.00	1.00	150
24	1.00	0.97	0.98	90
25	0.92	0.99	0.95	480
26	0.94	0.98	0.96	180
27	0.77	0.50	0.61	60
28	0.99	0.98	0.99	150
29	0.77	1.00	0.87	90
30	0.81	0.74	0.77	150
31	0.99	1.00	0.99	270
32	0.98	1.00	0.99	60
33	1.00	1.00	1.00	210
34	0.99	1.00	1.00	120
35	0.99	0.99	0.99	390
36	0.98	1.00	0.99	120
37	0.98	0.97	0.97	60
38	0.99	0.99	0.99	690
39	0.96	0.98	0.97	90
40	0.72	0.97	0.82	90
41	0.98	1.00	0.99	60
42	1.00	0.99	0.99	90
accuracy			0.98	12630
macro avg	0.96	0.97	0.96	12630
weighted avg	0.98	0.98	0.98	12630

Predictions on Test Data

```
plt.figure(figsize = (25, 25))
```

```
start_index = 0
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    prediction = pred[start_index + i]
    actual = labels[start_index + i]
    col = 'g'
    if prediction != actual:
        col = 'r'
    plt.xlabel('Actual={} || Pred={}'.format(actual, prediction), color =
col)
    plt.imshow(X_test[start_index + i])
plt.show()
```




```
ls
```

```
from PIL import Image
```

```
from collections import Counter

try:
  image = cv2.imread("/content/drive/MyDrive/Traffic_Sign_Classifier/Te
st/00005.png")
  image_fromarray = Image.fromarray(image, 'RGB')
  resize_image = image_fromarray.resize((30,30))
  data.append(np.array(resize_image))
except:
  print("Error in " + img)
X_test = np.array(data)
X_test = X_test/255

predictions = model.predict(X_test)
classIndex = model.predict_classes(X_test)
probabilityValue =np.amax(predictions)
print(classIndex)
probabilityValue


frameWidth= 640         # CAMERA RESOLUTION
frameHeight = 480
brightness = 180
threshold = 0.75         # PROBABLITY THRESHOLD
font = cv2.FONT_HERSHEY_SIMPLEX

# SETUP THE VIDEO CAMERA
cap = cv2.VideoCapture(0)
cap.set(3, frameWidth)
cap.set(4, frameHeight)
cap.set(10, brightness)


def grayscale(img):
  img = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
  return img
def equalize(img):
  img =cv2.equalizeHist(img)
  return img
def preprocessing(img):
  img = grayscale(img)
  img = equalize(img)
  img = img/255
  return img
def getCalssName(classNo):
  if   classNo == 0: return 'Speed Limit 20 km/h'
  elif classNo == 1: return 'Speed Limit 30 km/h'
  elif classNo == 2: return 'Speed Limit 50 km/h'
  elif classNo == 3: return 'Speed Limit 60 km/h'
```

```
  elif classNo == 4: return 'Speed Limit 70 km/h'
  elif classNo == 5: return 'Speed Limit 80 km/h'
  elif classNo == 6: return 'End of Speed Limit 80 km/h'
  elif classNo == 7: return 'Speed Limit 100 km/h'
  elif classNo == 8: return 'Speed Limit 120 km/h'
  elif classNo == 9: return 'No passing'
  elif classNo == 10: return 'No passing for vechiles over 3.5 metric t
ons'
  elif classNo == 11: return 'Right-of-way at the next intersection'
  elif classNo == 12: return 'Priority road'
  elif classNo == 13: return 'Yield'
  elif classNo == 14: return 'Stop'
  elif classNo == 15: return 'No vechiles'
  elif classNo == 16: return 'Vechiles over 3.5 metric tons prohibited'
  elif classNo == 17: return 'No entry'
  elif classNo == 18: return 'General caution'
  elif classNo == 19: return 'Dangerous curve to the left'
  elif classNo == 20: return 'Dangerous curve to the right'
  elif classNo == 21: return 'Double curve'
  elif classNo == 22: return 'Bumpy road'
  elif classNo == 23: return 'Slippery road'
  elif classNo == 24: return 'Road narrows on the right'
  elif classNo == 25: return 'Road work'
  elif classNo == 26: return 'Traffic signals'
  elif classNo == 27: return 'Pedestrians'
  elif classNo == 28: return 'Children crossing'
  elif classNo == 29: return 'Bicycles crossing'
  elif classNo == 30: return 'Beware of ice/snow'
  elif classNo == 31: return 'Wild animals crossing'
  elif classNo == 32: return 'End of all speed and passing limits'
  elif classNo == 33: return 'Turn right ahead'
  elif classNo == 34: return 'Turn left ahead'
  elif classNo == 35: return 'Ahead only'
  elif classNo == 36: return 'Go straight or right'
  elif classNo == 37: return 'Go straight or left'
  elif classNo == 38: return 'Keep right'
  elif classNo == 39: return 'Keep left'
  elif classNo == 40: return 'Roundabout mandatory'
  elif classNo == 41: return 'End of no passing'
  elif classNo == 42: return 'End of no passing by vechiles over 3.5 me
tric tons'


success, imgOrignal = cap.read()

# PROCESS IMAGE
img = np.asarray(imgOrignal)
img = cv2.resize(img,(30, 30,3))
```

```
img = preprocessing(img)
cv2.imshow("Processed Image", img)
img = img.reshape(1, 30, 30, 1)
cv2.putText(imgOrignal, "CLASS: " , (20, 35), font, 0.75, (0, 0, 255),
2, cv2.LINE_AA)
cv2.putText(imgOrignal, "PROBABILITY: ", (20, 75), font, 0.75, (0, 0, 2
55), 2, cv2.LINE_AA)
# PREDICT IMAGE
predictions = model.predict(img)
classIndex = model.predict_classes(img)
probabilityValue =np.amax(predictions)
if probabilityValue > threshold:
#print(getCalssName(classIndex))
  cv2.putText(imgOrignal,str(classIndex)+" "+str(getCalssName(classInde
x)), (120, 35), font, 0.75, (0, 0, 255), 2, cv2.LINE_AA)
  cv2.putText(imgOrignal, str(round(probabilityValue*100,2) )+"%", (180
, 75), font, 0.75, (0, 0, 255), 2, cv2.LINE_AA)
  cv2.imshow("Result", imgOrignal)

if cv2.waitKey(1) and 0xFF == ord('q'):
  break
```

FUTURE SCOPE

Another direction for further research is to develop a real time traffic sign recognition system which captures a video by a camera mounted on the vehicle, detects and recognises the traffic signs in real time and gives the result to the driver within a sufficient time frame in order to take the right action.

The crucial issue in real time applications is the time spent to recognise the traffic sign.

This should be reduced to the 169 minimum by choosing the proper techniques for real time applications and by optimising the code. The methods presented in this thesis can be modified to fit the real time requirements

Conclusion and findings:

We learned how to perform traffic sign classification and recognition with Keras and Deep Learning.

To create our traffic sign classifier, we:

- Utilized the popular German Traffic Sign Recognition Benchmark (GTSRB) as our dataset.
- Implemented a Convolutional Neural Network using the Keras deep learning library.
- Trained the model on the GTSRB dataset, obtaining **a good accuracy**.
- Created a Python script that loads our trained model and then classifies new input images.

REFERENCES-

- https://pyimagesearch.com/2019/11/04/traffic-sign-classification-with-keras-and-deep-learning/
- https://towardsdatascience.com/building-a-road-sign-classifier-in-keras-764df99fdd6a
- https://github.com/liferlisiqi/Traffic-Sign-Classifier
- https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign
- https://github.com/DeepranjanG/Traffic_sign_classification
- https://youtu.be/Vtc64rPHZ6I
- https://youtu.be/tPYj3fFJGjk
- https://youtu.be/SWaYRyi0TTs